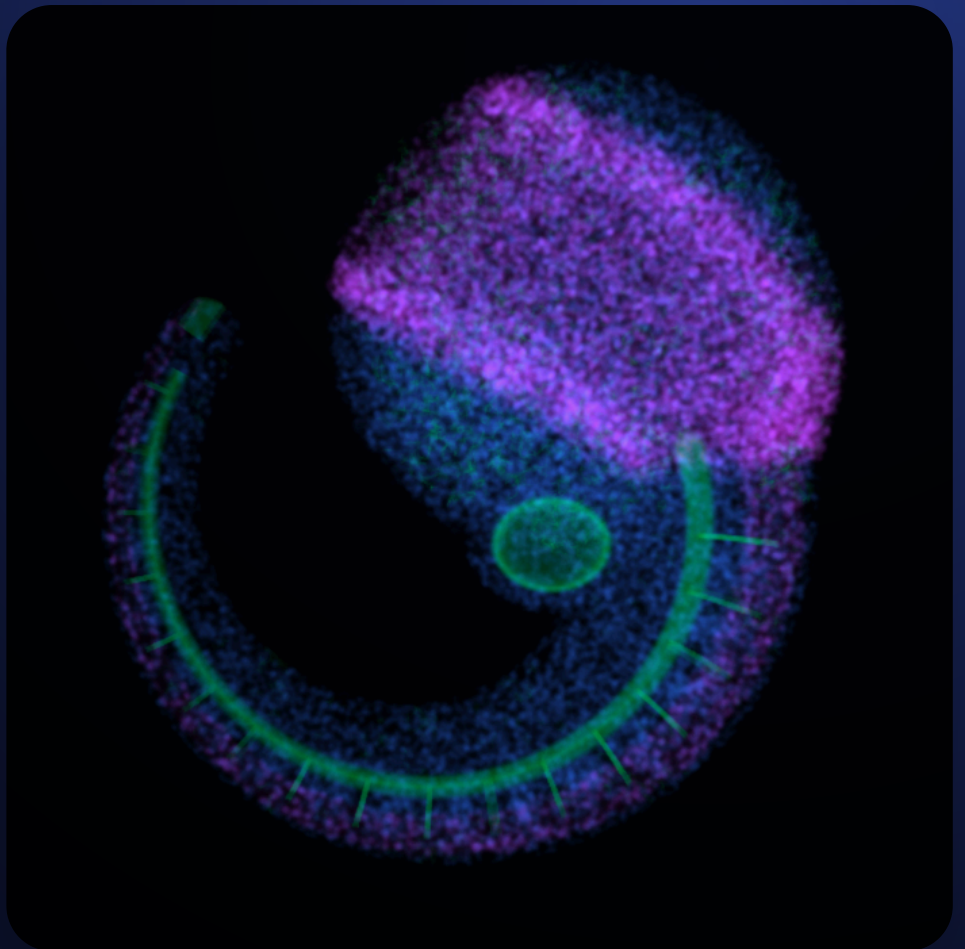




IRIBHM · ULB — LUMEN3D

Lumen3D — the essentials

What happens to your Imaris file, from the microscope to the screen



Ten pages to understand how an Imaris file is cleaned, reduced, cut into bricks and rendered in 3D in the browser — and what to keep in mind when you interpret your images. Each section points to the chapter of the full documentation that gives every detail.

Contents

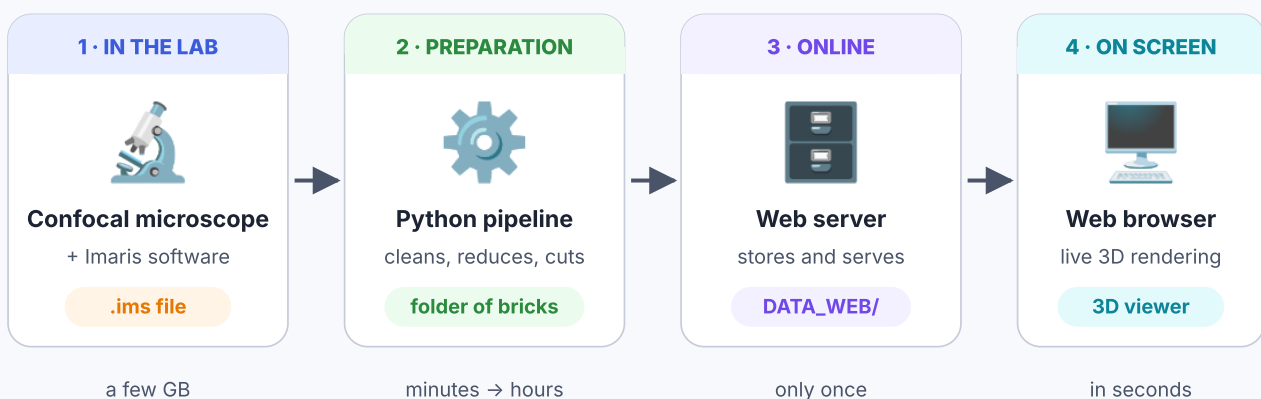
1. Lumen3D on one page	3
2. Your Imaris file: what is read	4
3. Cleaning: removing the background	4
3.1 How the background is removed	5
4. Reducing the size without loss	6
5. Publication	7
6. On screen: the 3D viewer	8
7. Tuning how a channel looks	9
8. Measuring, slicing, exporting	10
9. Keep in mind when interpreting your images	
Frequently asked questions	12

1. Lumen3D on one page

🕒 IN 30 SECONDS

- Lumen3D displays confocal volumes of **several gigabytes** in a plain web browser, with nothing to install.
- A **pipeline** prepares each Imaris file **once**; the original file is never modified.
- The **viewer** downloads only what the screen needs, then computes the 3D image on the graphics card.

The journey of an acquisition: from the microscope to the screen



The original file is never modified: the pipeline makes a web-optimised copy of it.

▲ The journey of an acquisition, in four steps.

3D

A confocal volume: a stack of optical sections, up to 4 channels displayed.

Live

A time series (timelapse); it may carry **cell tracking**.

2D

A calibrated photograph (for instance an X-gal staining).

i GOOD TO KNOW

This document is a summary. The → **chapter N** references point to the **full documentation** ("Lumen3D from A to Z"), published in the same place. The volume pictures come from a **synthetic demonstration embryo**.

2. Your Imaris file: what is read

⌚ IN 30 SECONDS

- An `.ims` file is a **binder** (HDF5 format) that holds the image and its information.
- The pipeline reads the **dimensions**, the **voxel size in μm** , the **channel names** and the **time stamps**.
- Only the full-resolution image is used; the calibration is **never invented**.

What is read

INFORMATION	EXAMPLE (DEMONSTRATION DATASET)
Voxels X × Y × Z	768 × 576 × 112
Physical extent	921.6 × 691.2 × 336 μm
Voxel size	1.2 × 1.2 × 3.0 μm
Channels	DAPI, Pecam1, Sox2
Timepoints	1 (3D) or n (Live)

1 2 3 4 WORKED EXAMPLE

The **voxel size** is not copied, it is **computed**:
extent ÷ number of voxels.

$921.6 \mu\text{m} \div 768 \text{ voxels} = 1.2 \mu\text{m}$ per voxel along X.

nm and mm are converted to μm . If the extent is missing, the dataset is marked “uncalibrated”: no scale bar, no measurement in μm .

👉 TIP

The file name matters. Stage and embryo are read from it: `...-E85-Em1-...` → stage **E8.5**, embryo **Em1**; `E8-5` and `E8.5` also give **E8.5**; `E105` gives **E10.5**. You can correct these values afterwards in the administration panel.

📖 GOING FURTHER

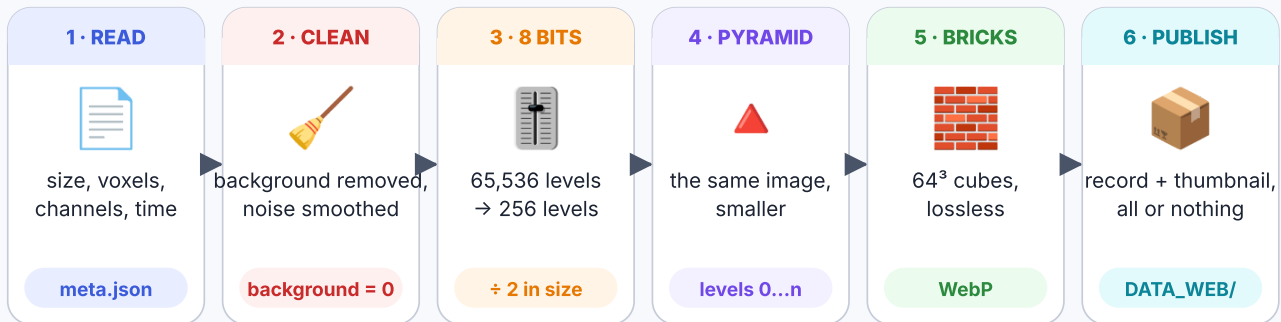
→ chapter 4 (voxels, channels, the structure of an `.ims`) and chapter 8 (reading the file name).

3. Cleaning: removing the background

⌚ IN 30 SECONDS

- The **noise floor** is measured in the **8 corners** of the volume, where there is no embryo.
- Around the signal nothing is touched; **outside the signal**, noise is smoothed (median filter).
- The image is then converted to **8 bits** by a simple rule of three: the background becomes **0**.

The six pipeline steps, for one .ims file



The original .ims file is never modified. Each step has its own chapter in the full documentation (4 to 8).

▲ The six pipeline steps for one file.

3.1 How the background is removed

- 1 Background floor:** 8 small cubes (up to $32 \times 32 \times 32$ voxels) are taken at the 8 corners of the volume. The floor = the value below which **99 %** of these corner voxels lie.
- 2 Signal ceiling:** the value below which **99.9 %** of the voxels lie (one voxel out of 4 in each direction is examined).
- 3 Signal mask:** every voxel above **$1.1 \times$ the floor**. Isolated hot pixels are removed (opening), then the mask is **grown by 3 voxels** to protect the natural halo of the cells.
- 4 Outside the mask only:** each voxel is replaced by the **median** of its $3 \times 3 \times 3$ neighbourhood. Inside the mask, voxels keep **exactly** their original value.
- 5 Conversion to 8 bits:** a straight line from the floor ($\rightarrow 0$) to the ceiling ($\rightarrow 255$).

Steps 2 + 3: from the raw signal (16 bits) to the displayed image (8 bits)



▲ Conversion to 8 bits: a rule of three, no gamma.

WORKED EXAMPLE

Floor = 4,000, ceiling = 34,000 (raw values). A raw voxel of **19,000** becomes $255 \times (19,000 - 4,000) \div (34,000 - 4,000) = 127$ (the decimal part is dropped). A voxel at 3,500 (below the floor) becomes **0**; a voxel at 40,000 becomes **255**.

? WHY THIS CHOICE?

An automatic thresholding method (Otsu) and a neural-network denoiser (Noise2Void) were tried and then **removed** (version 0.12.0): they created artificial coloured “blobs”. The current method only removes the camera background.

GOING FURTHER

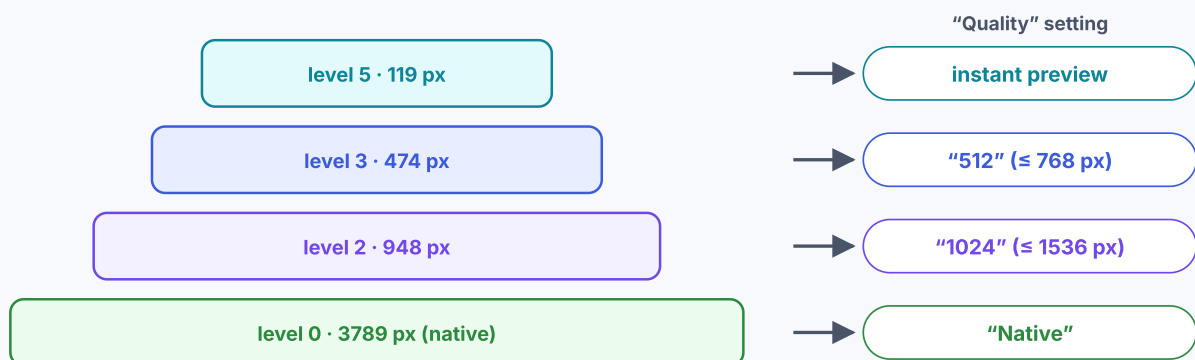
→ chapter 5: each step shown on a real slice, the shared window of timelapses, the tiling of very large volumes.

4. Reducing the size without loss

IN 30 SECONDS

- **8 bits** instead of 16: half the bytes.
- A **pyramid** of smaller and smaller versions shows the volume quickly, then refines it.
- The volume is cut into **bricks** of $64 \times 64 \times 64$ voxels, compressed **losslessly** (WebP); empty bricks are **not stored**.

The pyramid of a large stack (3789×3789 px) and the “Quality” setting



Each level = mean of 2×2 blocks ($\times 2$ in Z when possible) of the level below. Levels 1 and 4 exist too.

- ▲ The resolution pyramid of a large stack and the viewer’s “Quality” setting: “512” picks the finest level whose longest side is at most 768 px.

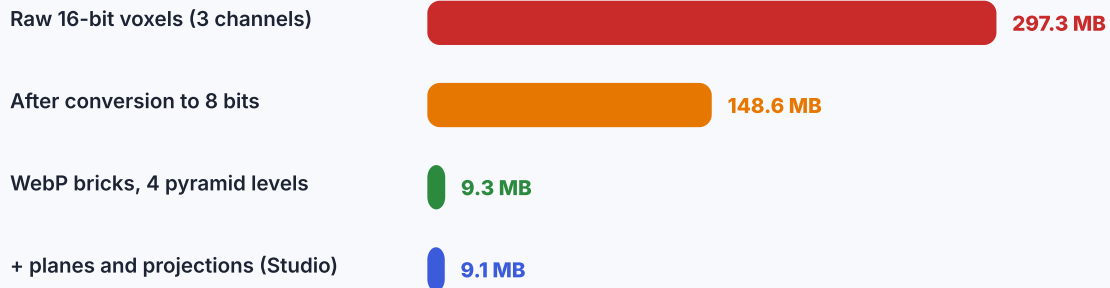
💡 A PICTURE TO UNDERSTAND

Like an online map: you never download the whole world, only the visible **tiles**, blurry first and then sharp. Bricks are the tiles of a volume.

📐 WORKED EXAMPLE

A block of 2×2 voxels worth 10, 11, 12 and 14 becomes, one level up, $(10 + 11 + 12 + 14 + 2) \div 4 = \mathbf{12}$ (integer mean, rounded).

The demonstration dataset: from 297 MB of voxels to 9 MB of bricks



Big win: the background becomes exactly 0, empty bricks are not stored and the rest compresses losslessly.

▲ What the demonstration dataset weighs at each step.

? WHY THIS CHOICE?

Lossless, because these are measurements: a lossy format (JPEG) would change the intensities. **WebP** rather than PNG because it is more compact at identical quality and the browser decodes it very fast. Each brick is laid flat as **one image** (its 66 slices arranged in a 9×8 grid, one-voxel border included).

📖 GOING FURTHER

→ chapter 6 (pyramid, compression, empty bricks) and chapter 7 (bricks, mosaics, packs, planes for fast slices).

5. Publication

⌚ IN 30 SECONDS

- The pipeline writes a **record** (`metadata.json`): dimensions, calibration, channels, stage.
- For a timelapse, it attaches the **cell tracking** when it finds one.
- The dataset is built aside and published **in one go**: never a half-copied dataset.

PUBLISHED FILE	ROLE
<code>metadata.json</code>	the dataset record (name, stage, channels, μm ...)
<code>thumbnail.webp</code>	the explorer thumbnail (false-colour maximum projection)
<code>bricks/</code>	the bricks of every level + their index
<code>planes/</code> , <code>mips/</code>	XY planes and per-layer projections, for the Studio
<code>tracks.json</code> , <code>model.glb</code>	cell tracking and the embryo surface (timelapses)
<code>download/</code>	(option) the original <code>.ims</code> , an ImageJ TIFF, PNG projections

GOOD TO KNOW

Cell tracking. The pipeline looks, in this order, for: a `.imaris_track` file, the *Spots/Tracks* objects inside the `.ims` itself, then the Imaris Excel export. Positions are **stabilised**: a rigid motion (rotation + translation, no deformation) cancels the drift of the embryo between two timepoints.

TIP

To put a dataset online: copy the produced folder into the server's `DATA_WEB/` , **or** drag it into the **Import** tab of the administration panel (resumable, verified transfer). An imported dataset stays **hidden** until you show it. If you re-process a dataset, your settings (name, colours, orientation...) are kept.

GOING FURTHER

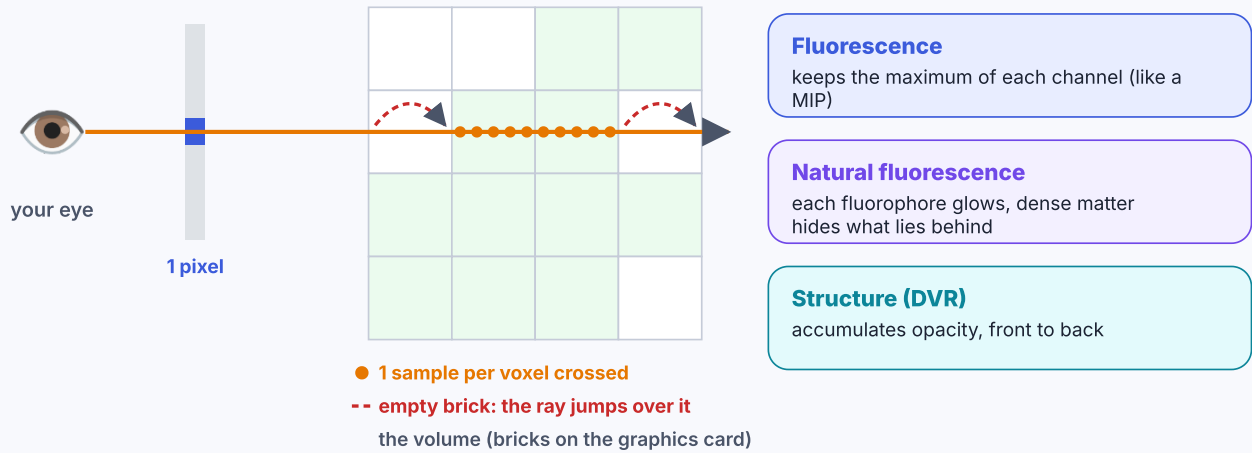
→ chapter 8, and the Administrator guide (Datasets and Import tabs).

6. On screen: the 3D viewer

IN 30 SECONDS

- The viewer first loads the **coarsest level** (instant picture), then the requested level, **starting from the centre**.
- The **512 / 1024 / Native** choice selects the pyramid level; if the graphics card lacks memory, a lighter level is used and a message says so.
- The image is **computed** for every pixel by casting a ray through the volume.

Ray marching: one computation per pixel, redone at every move



▲ Ray marching and the three render modes.

MODE	WHAT YOU SEE	GOOD FOR...
Fluorescence (default)	the maximum of each channel along the ray, colours added	spotting all the signal, like a MIP
Natural fluorescence	each fluorophore glows; dense matter hides what lies behind	perceiving depth and shapes
Structure (DVR)	opacity accumulated from front to back	surfaces and solid volumes

💡 A PICTURE TO UNDERSTAND

The processor is a head chef; the **graphics card** is a team of thousands of kitchen assistants who all make the same small gesture at the same time. Computing a pixel = one gesture: this is why the 3D image can be recomputed dozens of times per second.

📖 GOING FURTHER

→ chapter 9 (Three.js, ray marching, the formulas of each mode) and chapter 10 (streaming, graphics-card memory, timelapses).

7. Tuning how a channel looks

⌚ IN 30 SECONDS

- Each channel has a **min**, a **max**, a **gamma**, an **opacity** and a **colour**; channels are **added**.
- The histogram shows how the **0–255** values of the coarsest level are spread (64 bars).
- These settings change **only the display**, never the data.

Tuning a channel: min, max, gamma — example of a voxel of value 120



These settings only change the display: the data on the server are never modified.

▲ How one voxel is computed, from storage to the screen.

The histogram handles

- **Min / Max:** everything below *min* turns black, above *max* full colour.
- **Middle handle:** the level displayed at **50 %**; it sets the gamma ($\text{gamma} = \ln 0.5 \div \ln m$).
- **Auto:** clips the darkest 0.5 % and the brightest 0.5 %.

⚠ WARNING

Before your settings, the viewer also sets the weakest values of a channel to **0** (a “floor” between 6 and 48 estimated from the histogram). And at most **4 channels** are displayed at once.

📖 GOING FURTHER

→ chapter 11 (transfer function, histograms, Gaussian filter, colour-vision simulation).

8. Measuring, slicing, exporting

🕒 IN 30 SECONDS

- **Measurements** use the real voxel size: the result is in **µm**, even when voxels are longer in Z.
- **Oblique slice, Z-stack browser, Studio** and the **Compare** page produce publication-ready figures.
- Everything exports: an image at any size, a Studio figure, measurements as CSV, a sharing link.

1-2-4 WORKED EXAMPLE

Two points 10 voxels apart in X and 4 voxels apart in Z, with $1.2 \times 1.2 \times 3$ µm voxels: $\sqrt{(10 \times 1.2)^2 + (4 \times 3)^2} = \sqrt{144 + 144} = \mathbf{17\ \mu m}$ — not $\sqrt{10^2 + 4^2} \approx 10.8$ “voxels”.

TOOL	WHAT FOR
Measure distance	click two points on the volume; the point lands on the first visible structure along the ray (a click in empty space is refused)
Oblique slice	see the volume cut along any plane, with an exact scale bar
Z-stack browser	browse the sections from above, set the thickness, trim top and bottom
Studio	compose a figure (arrows, text, scale bar, distances) at full resolution
Compare	up to 4 datasets side by side, synchronised (camera, time, slices, channels)
Cell tracking (Live)	trajectories, surface, cell sheet, charts, cell-to-cell distances

WARNING

The **scale bar of the 3D view** is exact only at the depth of the **centre of the specimen** (perspective view: what is closer looks bigger). For a precise measurement, use the measuring tool or a slice.

GOING FURTHER

→ chapter 12 (every tool illustrated, keyboard shortcuts, the 2D page and X-gal stain isolation).

9. Keep in mind when interpreting your images

REMEMBER

- **Grey levels are relative.** Each channel of each dataset has its own window (floor → ceiling): do not compare an intensity between two channels or two datasets. For a timelapse, the window is **shared by all timepoints**: a drop in intensity (photobleaching) stays visible, as in reality.
- **Outside the signal, noise is smoothed** ($3 \times 3 \times 3$ median); **inside the signal**, nothing is changed.
- **8 bits** are enough to see morphology; to quantify, go back to the original `.ims` (downloadable from [download/](#) when that option was enabled).
- **Empty bricks** (no non-zero voxel after cleaning) are not stored: these areas really are 0, no data was lost.
- **Calibration**: without a voxel size in the file, no measurement in μm is offered.

Frequently asked questions

QUESTION	SHORT ANSWER
My dataset looks blurry at first	Normal: the coarse level arrives first, detail follows (progress bar).
"Native" gives me another level	The graphics card lacks memory: a lighter level is shown, with a message.
My colours changed	Channel settings are made in the viewer; default colours are set in the administration panel.
The dataset is not in the explorer	It may be hidden or still importing: see the Administrator guide.
Can I share a view?	Yes: the page link keeps the state of the view; the Compare page can also save a workspace.

GOING FURTHER

Full documentation: 15 chapters, about 100 pages, every step illustrated. **Administrator guide:** every screen of the panel, step by step.